Artificial Intelligence

Project 1

Nazia Islam

Part 0 - Setup your Environments [10 points]:

- **Code**: gridworld_lab.py
- **Requirements**: pip install matplotlib numpy

- **Explanation of how I generate the code**:
  The code has 4 main parts:

  1. Generate mazes: First, it has a constant part: grid size: 51, number of worlds: 30, and blocking probability: 30 percent of cells are blocked. These control everything; I can change them to make bigger/smaller grids or more/fewer worlds.
  2. Save/Load them: gridworlds_30.json stores all grid arrays so you can reload them later for experiments and gridworld_01.png: visual images of each maze.

     It asks "Load existing? [Y/n]" — if Y, it reads from JSON instead of regenerating.

  3. Draw them: it uses matplotlib's to render the grid — blocked cells appear dark, unblocked cells appear light.
  4. An interactive viewer to browse them: This is the GUI window with buttons

Part 1 - Understanding the methods [10 points]:

a.Explain in your report why the first move of the agent for the example search problem from Figure 8 is to the east rather than the north given that the agent does not know initially which cells are blocked.

Answer: The Agent is at E2, Target is at E5. The agent can't see any blocked cells yet, except its immediate neighbors. A* picks the cell with the lowest f-value = g + h
For the agent at E2, it has two options:

Move	g (cost so far)	h (Manhattan to T)	f = g+h
Go NORTH to D2	1	4	5
Go EAST to E3	1	2	3 (smaller)

Going East (E3**)** has a lower f-value because it moves directly toward the target, reducing the Manhattan distance faster. Going North moves away from row E, where the target sits, increasing the heuristic estimate.

Since the agent assumes all unseen cells are unblocked, East appears to be a shorter path. A* always picks the lowest f-value, so it moves East first.

b. This project argues that the agent is guaranteed to reach the target if it is not separated from it by blocked cells. Give a convincing argument that the agent in finite gridworlds indeed either reaches the target or discovers that this is impossible in finite time. Prove that the number of moves of the agent until it reaches the target or discovers that this is impossible is bounded from above by the number of unblocked cells squared.

Answer:
Case 1: **Agent finds the target when it is reachable**:
The agent uses the freespace assumption: it always tries the shortest presumed-unblocked path. If the target is truly reachable, there exists at least one real unblocked path from the agent to the target. Every time the agent replans; it finds a path through cells it currently believes are unblocked. Since blocked cells are never forgotten (the agent's knowledge only grows), every replan uses more accurate information than the last. The agent cannot get permanently stuck because if its current path is blocked, replanning will eventually discover the true unblocked path. Therefore, the agent must eventually reach the target.

Case 2: **Agent reports impossible when the target is unreachable**:
The agent only reports failure when A* finds that no presumed-unblocked path exists. As the agent explores, it gradually learns about more blocked cells. Since blocked cells are never forgotten, the agent's known map becomes increasingly accurate. If the target is truly unreachable, eventually the agent will have observed enough blocked cells that no path exists in its known map either. At that point, A* returns null → agent correctly reports "cannot reach target".
The agent never falsely reports failure when a path exists because it treats unknown cells as unblocked.

Case 3: **Upper bound when target is reachable**:

Total moves $\leq U^2$ where U = number of unblocked cells.

Every time the agent stops and replans, it is because it discovered at least one new blocked cell. Since there are at most U cells in total, this can happen at most U times.

Each path the agent follows is a shortest presumed-unblocked path; it never revisits the same cell twice. Since the grid has only U unblocked cells, any such path is at most U steps long.

Case 4: **Upper bound when the target is Unreachable**:

The agent declares failure only when A* finds no path at all in its current known map. Since blocked cells are never forgotten, the agent's knowledge keeps growing until eventually the blocked boundary around the target becomes fully visible, at which point A* returns null and the agent stops.

Just like Case 3, every replan is triggered by discovering at least one new blocked cell. There are at most U cells to discover, so the agent replans at most U times.

Before each replan, the agent follows a shortest path with no repeated cells, so each path is at most U steps long**.**

Part 2 - Implement repeated forward A* [20 points]:

Code: repeated_forward_astar.py

Run: python3 repeated_forward_astar.py

Part 3 - The Effects of Ties [10 points]:

Code: part3_ties.py

Run: python3 part3_ties.py

Observations and Explanation:

```
...@ada: ~/GPULLM/ParallelCodeEstimation/src/histogram-cuda — -zsh    ~ — n_i34@ada: ~/GPULLM/benchmarker — -zsh    ~ — n_i34@ada: ~/GPULLM/benchmarker — -zsh    ~/Desktop/AI/Project_1_final — -zsh

| | Large-g | Small-g |
|---|---|---|
| Expanded | 117 | 117 |
| Moves | 20 | 20 |
| Searches | 4 | 4 |
| Runtime (ms) | 3.24 | 2.08 |

  Launching side-by-side visualizer...

  Done!
(base) naziaislam@MacBookPro Project_1_final % python3 part3_ties.py
Loading worlds from gridworlds/gridworlds_30.json...
  Loaded 30 worlds.

==================================================
  PART 3 — THE EFFECTS OF TIES
==================================================

  Options:
  1. Visualize step-by-step on one world
  2. Run comparison on ALL 30 worlds + chart
  3. Both

  Enter choice [1/2/3]: 2

======================================================================
  COMPARISON: Large-g vs Small-g across all 30 worlds
======================================================================
  World |   LG-Exp |   SG-Exp | LG-Moves | SG-Moves |   LG-ms |   SG-ms |   Winner
  ----------------------------------------------------------------------
      1 |      117 |      117 |       20 |       20 |     2.5 |     2.0 | Tie
      2 |    1,920 |    2,510 |       73 |       99 |    22.1 |    17.7 | Large-g ✅
      3 |    1,231 |    1,231 |       40 |       40 |     8.0 |     6.7 | Tie
      4 |    1,744 |    1,549 |       63 |       55 |    10.3 |     7.6 | Small-g
      5 |       53 |       53 |       13 |       13 |     0.3 |     0.3 | Tie
      6 |      277 |      277 |       46 |       46 |     1.6 |     1.6 | Tie
      7 |      462 |      462 |       27 |       27 |     3.3 |     2.2 | Tie
      8 |   12,256 |   10,666 |      249 |      239 |    60.7 |    48.6 | Small-g
      9 |    4,081 |    4,081 |       96 |       96 |    21.7 |    19.4 | Tie
     10 |       81 |       81 |       18 |       18 |     0.5 |     0.5 | Tie
     11 |    2,031 |    2,031 |       91 |       91 |    11.2 |    10.0 | Tie
     12 |    1,012 |    1,012 |       72 |       72 |     5.5 |     5.0 | Tie
     13 |    2,498 |    2,498 |       87 |       87 |    13.3 |    12.1 | Tie
     14 |    6,168 |    6,499 |      117 |      103 |    30.3 |    29.1 | Large-g ✅
     15 |    2,976 |    2,976 |       51 |       51 |    15.7 |    13.6 | Tie
     16 |    1,404 |    1,404 |       72 |       72 |     7.7 |     7.1 | Tie
     17 |    1,084 |    1,084 |       71 |       71 |     5.9 |     5.5 | Tie
     18 |    2,794 |    2,794 |      113 |      113 |    15.0 |    14.3 | Tie
     19 |      119 |      119 |       27 |       27 |     0.8 |     0.8 | Tie
     20 |    4,832 |    6,025 |       79 |       83 |    25.2 |    27.7 | Large-g ✅
     21 |    1,188 |    1,188 |       59 |       59 |     6.2 |     5.6 | Tie
     22 |    3,168 |    3,168 |       64 |       64 |    16.4 |    15.2 | Tie
     23 |      627 |      722 |       48 |       62 |     3.4 |     3.6 | Large-g ✅
     24 |    1,182 |    1,179 |       73 |       71 |     6.8 |     6.4 | Small-g
     25 |    1,290 |    1,290 |       87 |       87 |     7.1 |     6.4 | Tie
     26 |   23,031 |   23,031 |      111 |      111 |   118.7 |   101.7 | Tie
     27 |    4,969 |    5,002 |       94 |       98 |    26.4 |    23.5 | Large-g ✅
     28 |    7,869 |    7,869 |      141 |      141 |    41.1 |    45.0 | Tie
     29 |      158 |      158 |       28 |       28 |     0.9 |     0.9 | Tie
     30 |    5,025 |    5,025 |       76 |       76 |    25.3 |    23.3 | Tie
  ----------------------------------------------------------------------
    AVG |    3,187 |    3,202 |       73 |       74 |    17.1 |    15.5 |

  Large-g wins: 5/30   Small-g wins: 3/30   Ties: 22/30
  Large-g avg expanded: 3,187  vs  Small-g avg expanded: 3,202
  Large-g expands 0.5% FEWER cells on average
======================================================================


  Done!
(base) naziaislam@MacBookPro Project_1_final %
(base) naziaislam@MacBookPro Project_1_final %
```

Large-g expands 0.5% fewer cells on average (3,187 vs 3,202). Large-g wins 5 worlds, Small-g wins only 3, and 22 are ties.

In our DFS-generated maze environments, Large-g tie-breaking expands slightly fewer cells than Small-g. However, the advantage is small because our corridor-heavy mazes rarely produce many ties: most worlds produce identical results regardless of tie-breaking strategy. The theoretical advantage of Large-g is most visible in open gridworlds with few obstacles, where many cells share f-values and tie-breaking determines the entire search direction. In practice, Small-g occasionally runs faster in wall-clock time despite expanding more cells, due to simpler priority queue ordering. Overall, Large g is the better choice when the gridworld has open regions, while in dense mazes, the choice makes little practical difference.

Part 4: Forward A* vs. Backward A* [10 points]:

Code: part4_forward_vs_backward

```
  Large-g wins: 5/30   Small-g wins: 3/30   Ties: 22/30
  Large-g avg expanded: 3,187  vs  Small-g avg expanded: 3,202
  Large-g expands 0.5% FEWER cells on average
======================================================================


  Done!
(base) naziaislam@MacBookPro Project_1_final %
(base) naziaislam@MacBookPro Project_1_final % python3 part4_forward_vs_backward.py
Loading worlds from gridworlds/gridworlds_30.json...
  Loaded 30 worlds.

==============================================
  PART 4 — FORWARD A* vs BACKWARD A*
==============================================

  Options:
  1. Visualize step-by-step on one world
  2. Run comparison on ALL 30 worlds + chart
  3. Both

  Enter choice [1/2/3]: 2

======================================================================
  COMPARISON: Forward A* vs Backward A* across all 30 worlds
======================================================================
  World |  FWD-Exp |  BWD-Exp |  FWD-Mv |  BWD-Mv |  FWD-ms |  BWD-ms |     Winner
  ----------------------------------------------------------------------
      1 |      117 |      148 |       20 |       20 |      2.5 |      8.5 | Forward ✓
      2 |    1,920 |    3,968 |       73 |       97 |     20.0 |     40.0 | Forward ✓
      3 |    1,231 |    1,401 |       40 |       40 |      6.6 |     12.2 | Forward ✓
      4 |    1,744 |    2,056 |       63 |       55 |      9.3 |     17.3 | Forward ✓
      5 |       53 |       64 |       13 |       13 |      0.4 |      1.9 | Forward ✓
      6 |      277 |      279 |       46 |       44 |      1.5 |      7.2 | Forward ✓
      7 |      452 |      475 |       27 |       27 |      2.4 |      5.9 | Forward ✓
      8 |   12,256 |   15,504 |      249 |      255 |     59.4 |    104.4 | Forward ✓
      9 |    4,081 |    6,997 |       96 |      100 |     21.3 |     50.1 | Forward ✓
     10 |       81 |       82 |       18 |       18 |      0.5 |      1.9 | Forward ✓
     11 |    2,031 |    3,185 |       91 |       93 |     11.0 |     29.1 | Forward ✓
     12 |    1,012 |    1,266 |       72 |       72 |      5.7 |     14.9 | Forward ✓
     13 |    2,498 |    3,870 |       87 |       87 |     12.8 |     32.9 | Forward ✓
     14 |    6,168 |    4,437 |      117 |      103 |     29.8 |     38.6 | Backward ✓
     15 |    2,976 |    3,165 |       51 |       51 |     15.6 |     23.6 | Forward ✓
     16 |    1,404 |    1,260 |       72 |       58 |      7.8 |     14.1 | Backward ✓
     17 |    1,084 |    2,396 |       71 |      105 |      5.9 |     25.9 | Forward ✓
     18 |    2,794 |    4,186 |      113 |       93 |     14.7 |     37.2 | Forward ✓
     19 |      119 |      194 |       27 |       29 |      0.8 |      4.1 | Forward ✓
     20 |    4,832 |    5,959 |       79 |       83 |     25.1 |     39.1 | Forward ✓
     21 |    1,188 |    1,343 |       59 |       59 |      6.2 |     14.7 | Forward ✓
     22 |    3,168 |    3,489 |       64 |       62 |     16.8 |     28.8 | Forward ✓
     23 |      627 |    1,024 |       48 |       62 |      3.3 |     13.4 | Forward ✓
     24 |    1,182 |    1,642 |       73 |       71 |      6.7 |     18.2 | Forward ✓
     25 |    1,298 |    1,498 |       87 |       87 |      7.2 |     17.0 | Forward ✓
     26 |   23,031 |   23,573 |      111 |      111 |    118.8 |    143.3 | Forward ✓
     27 |    4,969 |   10,824 |       94 |      108 |     26.6 |     71.7 | Forward ✓
     28 |    7,869 |   11,562 |      141 |      143 |     40.5 |     87.7 | Forward ✓
     29 |      150 |      273 |       28 |       28 |      0.9 |      4.4 | Forward ✓
     30 |    5,025 |    6,229 |       76 |       76 |     25.6 |     45.0 | Forward ✓
  ----------------------------------------------------------------------
    AVG |    3,187 |    4,078 |       73 |       75 |     16.9 |     31.8 |

  Forward wins:  28/30
  Backward wins: 2/30
  Ties:          0/30

  Forward  avg expanded: 3,187
  Backward avg expanded: 4,078
  → Forward expands 21.8% FEWER cells on average
======================================================================

  Done!
(base) naziaislam@MacBookPro Project_1_final %
```

Explanation and Observation:

Forward A* significantly outperforms Backward A* across all 30 mazes, winning 28 out of 30 comparisons and expanding 21.8% fewer cells on average. The key reason is that Forward A* searches from the agent toward the fixed target since the target never moves, its Manhattan distance h-values remain perfectly valid across all replanning steps. Backward A* searches from the target toward the agent, but the agent moves after every replanning step, forcing h-values to be completely recomputed before each search. This recompilation overhead, combined with less accurate intermediate h-values causes Backward A* to expand more cells and run nearly twice as slow. Backward A* only wins in rare cases where the maze structure around the target is more open than around the agent, allowing the backward search to avoid dead ends that the forward search encounters.

Part 5 - Adaptive A* vs. Forward A* [20 points]:

Code: part5_adaptive.py

```
...@ada: ~/GPULLM/ParallelCodeEstimation/src/histogram-cuda — -zsh    ~ — ri_i34@ada: ~/GPULLM/benchmarker — -zsh    ~ — ri_i34@ada: ~/GPULLM/benchmarker — -zsh    ~/Desktop/AI/Project_1_final — python3 part5_adaptive.py

| | Forward | Backward |
|---|---|---|
| Expanded | 117 | 148 |
| Moves | 20 | 20 |
| Searches | 4 | 5 |
| Runtime (ms) | 2.32 | 8.08 |

  Launching visualizer...

  Done!
(base) naziaislam@MacBookPro Project_1_final % python3 part5_adaptive.py
Loading worlds from gridworlds/gridworlds_30.json...
  Loaded 30 worlds.

==================================================
  PART 5 — ADAPTIVE A* vs FORWARD A*
==================================================

  Options:
  1. Visualize step-by-step on one world
  2. Run comparison on ALL 30 worlds + chart
  3. Both

  Enter choice [1/2/3]: 2

======================================================================
  COMPARISON: Forward A* vs Adaptive A* across all 30 worlds
======================================================================
  World |  FWD-Exp |  ADA-Exp |  FWD-Mv |  ADA-Mv |  FWD-ms |  ADA-ms |      Winner
  ----------------------------------------------------------------------
      1 |      117 |      117 |      20 |      20 |     2.4 |     2.5 | Tie
      2 |    1,920 |    1,906 |      73 |      73 |    22.4 |    16.4 | Adaptive ✓
      3 |    1,231 |    1,230 |      40 |      40 |     8.5 |     8.3 | Adaptive ✓
      4 |    1,744 |    1,547 |      63 |      55 |    10.3 |     9.1 | Adaptive ✓
      5 |       53 |       53 |      13 |      13 |     0.4 |     0.4 | Tie
      6 |      277 |      265 |      46 |      46 |     1.6 |     1.9 | Adaptive ✓
      7 |      452 |      451 |      27 |      27 |     2.5 |     2.5 | Adaptive ✓
      8 |   12,256 |    9,135 |     249 |     247 |    60.4 |    51.1 | Adaptive ✓
      9 |    4,081 |    4,068 |      96 |      96 |    21.7 |    22.8 | Adaptive ✓
     10 |       81 |       80 |      18 |      18 |     0.5 |     0.5 | Adaptive ✓
     11 |    2,031 |    2,012 |      91 |      91 |    11.1 |    11.7 | Adaptive ✓
     12 |    1,012 |      995 |      72 |      72 |     5.4 |     5.9 | Adaptive ✓
     13 |    2,498 |    2,431 |      87 |      87 |    13.2 |    13.5 | Adaptive ✓
     14 |    6,168 |    6,104 |     117 |     111 |    29.7 |    30.4 | Adaptive ✓
     15 |    2,976 |    2,976 |      51 |      51 |    15.6 |    16.5 | Tie
     16 |    1,404 |    1,366 |      72 |      72 |     7.8 |     8.0 | Adaptive ✓
     17 |    1,084 |    1,069 |      71 |      71 |     6.0 |     6.1 | Adaptive ✓
     18 |    2,794 |    2,746 |     113 |     113 |    15.0 |    16.1 | Adaptive ✓
     19 |      519 |      519 |      27 |      27 |     0.8 |     0.8 | Tie
     20 |    4,832 |    6,825 |      79 |      83 |    25.2 |    34.0 | Forward
     21 |    1,188 |    1,183 |      59 |      59 |     6.3 |     6.6 | Adaptive ✓
     22 |    3,168 |    3,168 |      64 |      64 |    16.3 |    17.6 | Tie
     23 |      627 |      617 |      48 |      48 |     3.3 |     3.4 | Adaptive ✓
     24 |    1,182 |    1,177 |      73 |      71 |     6.7 |     7.0 | Adaptive ✓
     25 |    1,290 |    1,277 |      87 |      87 |     6.8 |     7.3 | Adaptive ✓
     26 |   23,031 |   23,026 |     111 |     111 |   110.1 |   140.7 | Adaptive ✓
     27 |    4,969 |    4,998 |      94 |      98 |    27.2 |    29.8 | Forward
     28 |    7,869 |    7,840 |     141 |     141 |    43.4 |    44.3 | Adaptive ✓
     29 |      150 |      150 |      28 |      28 |     0.9 |     0.9 | Tie
     30 |    5,825 |    5,824 |      76 |      76 |    26.2 |    28.3 | Adaptive ✓
  ----------------------------------------------------------------------
    AVG |    3,187 |    3,104 |      73 |      73 |    17.2 |    18.1 |

  Adaptive wins:  22/30
  Forward wins:   2/30
  Ties:           6/30

  Forward  avg expanded: 3,187
  Adaptive avg expanded: 3,104
  → Adaptive expands 2.6% FEWER cells on average
======================================================================
```

Observation and Explanation:

Adaptive A* wins 22 out of 30 worlds and expands 2.6% fewer cells on average (3,104 vs 3,187).

Adaptive A* outperforms Forward A* in 22 out of 30 mazes by expanding 2.6% fewer cells on average. The advantage comes from updating h-values after each search using the formula h_new(s) = g(goal) - g(s). These updated values are always at least as large as the original Manhattan distances, making future searches more focused by skipping cells that were previously found to be far from the goal. The improvement is most visible in complex worlds requiring many replanning steps, where the accumulated h-value updates provide significant guidance. However, Adaptive A* runs slightly slower in wall-clock time (18.1 ms vs 17.2 ms) because the h-value update step adds overhead after every search. This overhead is small for 51×51 grids, but the cell expansion savings would grow significantly in larger gridworlds, making Adaptive A* increasingly beneficial as grid size increases.

Part 6 - Heuristics in the Adaptive A* [10 points]:

Proof 1: **Manhattan Distance is Consistent**

Let s = any cell at position (r1, c1)
Let s' = succ(s,a) = neighbor cell at (r2, c2) after one move
Let goal = target cell at (rt, ct)
$h(s) = |r1 - rt| + |c1 - ct|$
$c(s,a) = 1$ (all moves cost 1)

Condition 1:
$h(goal) = |rt - rt| + |ct - ct| = 0 + 0 = 0$

Condition 2:
Since the agent moves in 4 directions, one move changes either r by ±1 OR c by ±1
$h(s) = |r1 - rt| + |c1 - ct|$

After one move (r2=r1, c2=c1+1)
$h(s') = |r2 - rt| + |(c1+1) - ct|$

by triangle inequality:
$|c1 - ct| \leq 1 + |(c1+1) - ct|$

Therefore,
$h(s) \leq 1 + h(s') = c(s,a) + h(s')$

Proof 2: **Adaptive A* Keeps H-Values Consistent**

Let s and s' be two adjacent expanded cells
g(s) = shortest path cost from start to s (found by A*)
g(goal) = shortest path cost from start to goal
h_new(s) = g(goal) - g(s)

Condition 1:
h_new(goal) = g(goal) - g(goal) = 0

Condition 2:
$g(goal) - g(s) \leq c(s,a) + g(goal) - g(s')$

$g(s') \leq g(s) + c(s,a)$ {subtracting g(goal) from both sides)

Part 7 - Memory Issues [10 points]:

Step1: N × N cells,

Variable	Purpose	Naive size	Optimized size
grid	blocked/unblocked	1 byte (int8)	1 bit
Known	agent's observed map	1 byte (int8)	2 bits (-1/0/1)
g- value	path cost from start	8 bytes (float64)	4 bytes (int32)
h-value	heuristic estimate	8 bytes (float64)	2 bytes (int16)
search stamp	lazy init counter	4 bytes (int32)	4 bytes (int32)
tree-pointer	parent direction	8 bytes (pointer)	2 bits
open/closed	in heap or done	implicit in heap	1 bit

Step 2:

In a 4-direction grid, a cell's parent can only come from 4 possible directions:

North (00), West (10), East (11) and South (01)

Step 3: Memory Calculation:

Total cells = 1001 × 1001 = 1,002,001 cells

Naive Implementation

Variable	Bytes/cell	Total bytes	Total MB
grid	1	1,002,001	0.96 MB
Known	1	1,002,001	0.96 MB
g- value	8	8,016,008	7.65 MB
h-value	8	8,016,008	7.65 MB
search stamp	4	4,008,004	3.82 MB
tree-pointer	8	8,016,008	7.65 MB
Total	30	30,060,030	≈ 28.7 MB

Optimized Implementation:

Variable	Bytes/cell	Total bytes	Total MB
grid	1bit	0.125	0.12 MB
Known	2 bits	0.25	0.24 MB
g- value	int32	4	3.82 MB
h-value	int16	2	1.91MB
search stamp	int32	4	3.82 MB
tree-pointer	2 bits	0.25	0.24 MB
Total		≈ 10.625	≈ 10.15 MB

Step 4: Largest Grid Within 4 MB

With Optimized implementation:

Maximum cells in 4 MB:

4 MB = 4 × 1024 × 1024 = 4,194,304 bytes

Max cells = 4,194,304 ÷ 10.625 ≈ 394,993 cells

Grid side = √394,993 ≈ 628

➔ Largest grid ≈ 628 × 628

With naive implementation:

Max cells = 4,194,304 ÷ 30 ≈ 139,810 cells

Grid side = √139,810 ≈ 374

➔ Largest grid ≈ 374 × 374

STEP 5: Additional Ways to Save Memory: Don't store h-values at all

grid + known combined = 1.000 byte (3 bits packed into 1 byte)

g-value = 4.000 bytes (int32)

h-value = 0.000 bytes (computed live)

search stamp = 2.000 bytes (int16) t

tree-pointer = 0.250 bytes (2 bits)

TOTAL = 7.25 bits + 6 bytes ≈ 6.0 bytes per cell

Maximum optimized:

4 MB = 4 × 1024 × 1024 = 4,194,304 bytes

Max cells = 4,194,304 ÷ 6 = 699,050 cells

Grid side = √699,050 ≈ 836

➔ Largest grid ≈ 835 × 835